

Optimization of Propellant Masses and Launch Trajectory for General Launch Vehicle

Luke Cresswell¹

University of Alabama, Tuscaloosa, AL, 35401

This paper presents a numerical framework for optimizing propellant mass distribution and ascent trajectory for a two-stage launch vehicle with fixed engine parameters. The ascent problem is formulated as a nonlinear problem corresponding to various phases of ascent trajectory. This problem is solved via direct transcription for control variables including stage propellant masses, initial flight path angle, and second stage discretized thrust vector history. Results demonstrate the sensitivity of solutions to initial conditions and constraints, with multiple viable candidates for achieving a target orbit while varying in propellant masses. Although simplified, this model captures key ascent dynamics and provides a useful sense for design tradeoffs in launch vehicle designs.

I. Introduction

In 1903, Konstantin Tsiolkovsky published his famous ideal rocket equation in the Russian journal *Nauchnoye Obozreniye*, relating the velocity change possible by a chemical rocket to its exhaust velocity and mass ratio [1].

$$\Delta V = v_e \ln \left(\frac{m_0}{m_f} \right) \quad (1)$$

Modern orbital launch vehicles, as a direct consequence of the physics behind this equation, are almost entirely propellant. A typical rocket on the launch pad is roughly 90% propellant by mass, with payload accounting for as little as 2% of the total. For a given engine, achieving greater ΔV requires more propellant or lower dry mass, and the relationship is logarithmic. Every meter per second lost to drag becomes harder to recover. Each additional kilo of propellant translates to less and less growth in ΔV . Thus, it is vitally important to minimize propellant mass as well as gravity and drag losses.

The thrust produced by a rocket nozzle varies with altitude according to,

$$T = \dot{m}V_e + (P_e - P_{atm})A_e \quad (2)$$

At sea level, atmospheric backpressure reduces nozzle performance, favoring high thrust engines with modest expansion ratios. In vacuum, a fully optimized engine allows exhaust gases to expand more fully, extracting more energy from the flow and producing more thrust from less propellant. This is one of the factors that drives the staged architecture common to modern launch vehicles. A simplified model of this architecture involves a high thrust stage designed to operate in the lower atmosphere and quickly move the vehicle out of the atmosphere, and a vacuum stage to insert the payload into the desired orbit.

In order to reduce design and manufacturing costs, instead of designing a custom engine for each mission, launch engineers commonly start with a historical engine with tested and proven performance. Given fixed engine parameters, major design considerations involve the division of propellant between stages as well as the trajectory the rocket takes to orbit. This paper aims to present a numerical framework for estimating a solution to a simplification of this complicated design challenge. We aim to optimize, for a given set of engine and rocket parameters that can be set by the user, the propellant masses, launch angle, and second stage thrust vector history to achieve a target low Earth orbit with a certain payload mass while minimizing propellant consumed.

¹ Graduate Student, Department of Aerospace Engineering and Mechanics, The University of Alabama

II. System Models

Stage Model

To simplify our problem as a first approximation, we assume a 2-stage rocket. The first stage is a solid rocket booster that assumes strong passive stability and follows a gravity turn trajectory to minimize drag losses and to simplify the aerodynamic model. We additionally assume that the stage starts with the pitch over maneuver already concluded. Thus, the initial state starts at an altitude of 1km and a velocity that the user can set in the parameters file.

The second is a vacuum burn to insert the payload into orbit. Generally, there is a coast phase between these two, but we allow for the possibility that the solid rocket burns above the atmosphere, to give the optimizer room to maneuver the solution space. The vacuum stage is ignited as the rocket leaves the atmosphere.

Engine Model

We model our engine as ideal. We assume the flow is isentropic, propellants combust completely, and chamber pressure and temperature are constants. The user should input the combustion chamber's total temperature and pressure, the specific heat ratio and the gas constant of the propellants, and the physical dimensions of the nozzle (throat and exit areas). Additionally, the user should supply the mass of the engines.

Force Model

We model gravity as decreasing by distance from the earth's center, according to Newton's Law of Universal Gravitation.

$$\vec{g} = -\frac{GM_E}{r^2} \hat{r} \quad (3)$$

We model drag according to the standard equation,

$$\vec{D} = -\frac{1}{2} \rho V^2 C_d A_{ref} \hat{V} \quad (4)$$

The drag coefficient and reference areas are user supplied parameters.

Mass Model

In order to roughly estimate the initial mass of the launch vehicle, we relate the supporting structures (tanks, fuel lines, pumps, casing, pressurant, etc.) to the propellant mass through a simple mass fraction. The user can change this to approximate low support systems, such as solid systems, and high support systems, such as high-pressure liquid systems. The total mass is approximated as,

$$M_i = m_{payload} + m_{engine1} + m_{engine2} + (1 + f_1) * m_{solid} + (1 + f_2) * m_{liquid} \quad (5)$$

Atmosphere Model

We use a two-layer standard atmosphere exponential decay model, depicted below (**Fig. 1**). Within the troposphere (below 11 km), the temperature decreases with altitude at a constant lapse rate. Pressure follows from the hydrostatic equation combined with the ideal gas law

$$P(h) = P_{sl} \left(\frac{T(h)}{T_{sl}} \right)^{\frac{g_0}{R_{air}L}} \quad (6)$$

Above 11 km, the stratosphere is treated as isothermal at $T = 216.65$ K,

$$P(h) = P_{11} * \exp \left(-\frac{g_0(h - 11000)}{R_{air}T_{strat}} \right) \quad (7)$$

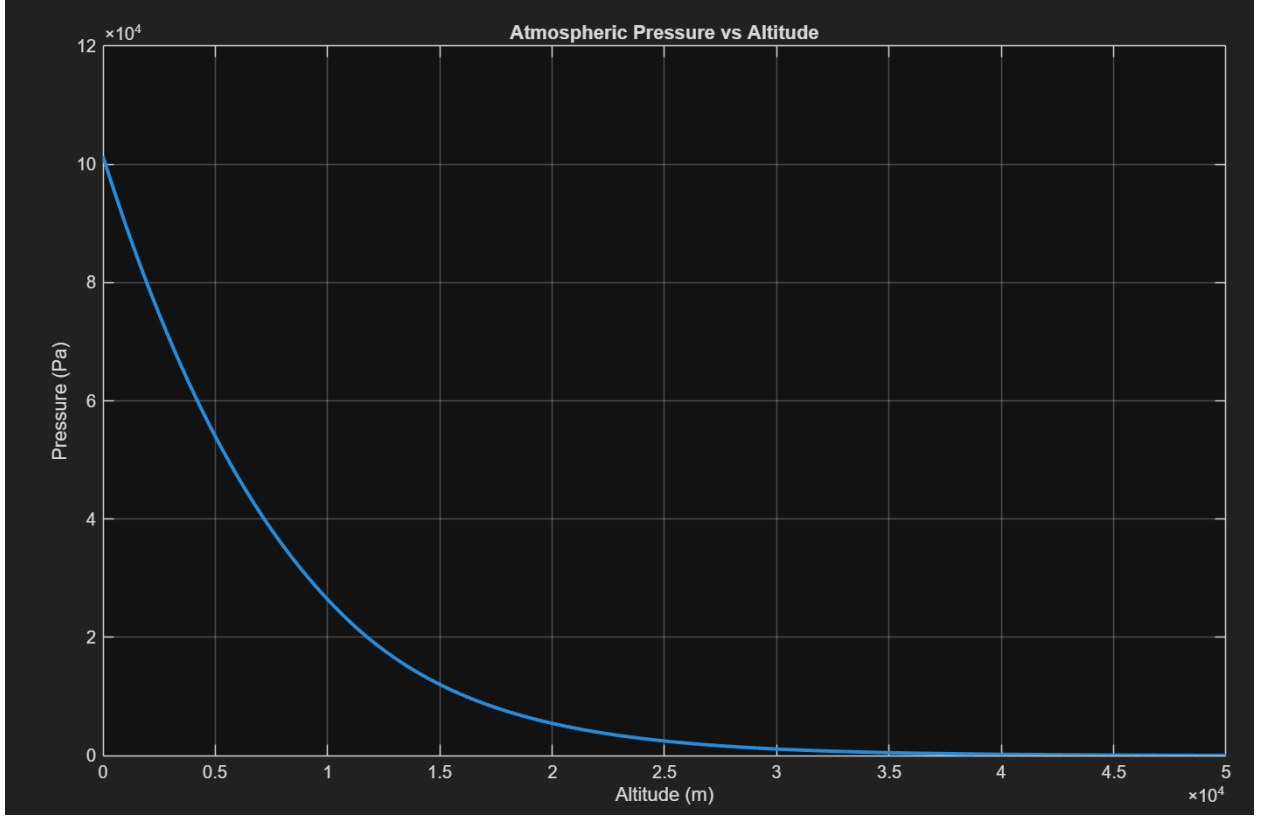


Figure 1: Atmospheric Model of Pressure from Two-Layer Standard Atmosphere Model

III. Equations of Motion

Our equations of motion are defined in a polar coordinate system (r- θ -h) centered at the center of the Earth to enable simpler connections between orbital mechanics and ascent path. We define r as the radially outward direction and θ as the direction perpendicular to r in the direction of the velocity vector. A simple cross product completes our definition of the dextral orthogonal triad. We define our thrust angle as δ , the angle from the velocity vector to the thrust vector, measured about $\hat{h}$. We note that this angle accounts for both the attitude of the rocket and its gimbaling, as the two can be combined since we are assuming our rocket is a point mass, and we are unconcerned with attitude dynamics. For our gravity turn, the only phase in atmosphere, $\delta=0$. In general, our differential equations are,

$$\ddot{r} = -\frac{T}{m} \sin(\delta - \gamma) - \frac{D}{m} \sin(\gamma) - g + r\dot{\theta}^2 \quad (8)$$

$$\ddot{\theta} = \frac{T}{mr} \cos(\delta - \gamma) - \frac{D}{mr} \cos(\gamma) - \frac{2\dot{r}\dot{\theta}}{r} \quad (9)$$

Where the velocities and accelerations are, from standard dynamics,

$$v_r = \dot{r} \quad (10)$$

$$v_\theta = r\dot{\theta} \quad (11)$$

$$a_r = \ddot{r} - r\dot{\theta}^2 \quad (12)$$

$$a_\theta = r\ddot{\theta} - 2\dot{r}\dot{\theta} \quad (13)$$

We define our state vector X as,

$$X = \begin{bmatrix} r \\ \theta \\ v_r \\ v_\theta \\ m \end{bmatrix} \quad (14)$$

For a complete derivation, see Appendix A.

IV. Numerical Method

The trajectory optimization is posed as a nonlinear program solved by direct transcription. We define our control vector as,

$$u = \begin{bmatrix} m_{solid} \\ m_{liquid} \\ \gamma_0 \\ \delta_1 \\ \vdots \\ \delta_N \end{bmatrix} \quad (15)$$

Where m_{solid} , m_{liquid} are the indicated propellant masses, γ_0 is the initial flight path angle or launch angle, and $\delta_1 - \delta_N$ are the discretized control history for the thrust vector. Our objective is to minimize the total propellant consumed.

$$\min J = m_s + m_l \quad (16)$$

The terminal state must satisfy equality constraints requiring the achieved orbit to match the target semi-major axis a^* and eccentricity e^* .

$$c_{eq,1} = \frac{a_f - a^*}{a^*} = 0 \quad (17)$$

$$c_{eq,2} = \frac{e_f - e^*}{0.1} = 0 \quad (18)$$

Where the denominators normalize each constraint to $O(1)$ to prevent the semi-major axis from dominating the optimizer. Additionally, the control variables are bound within a certain range to prevent the optimizer from exploring unfeasible solutions. The masses cannot be less than 30% of the initial guess, nor more than 800%. The initial flight path angle is constrained between $[55^\circ, 90^\circ]$. The thrust vector angle range is left as a user set range.

The thrust deflection history is parameterized over the normalized burn fraction, $\tau \in [0,1]$, since the start of the burn is variable and computationally expensive to predict.

$$\tau = \frac{t - t_3}{t_4 - t_3} \quad (19)$$

Where t_3 and t_4 correspond to the second stage ignition and burnout times, respectively. Each of the N control nodes are placed at uniform intervals within τ and are then linearly interpolated for burn fractions that do not directly correspond to a particular node. This additionally serves the purpose of keeping the positions of the node constant even when the burn length increases. The ODE evaluates $\delta(\tau)$ at each time step by mapping the current time to τ and interpolating.

Our ODE takes advantage of MATLAB's built in integrator and event functions [2]. By setting up a number of events, such as reaching the end of the atmosphere, we can determine what the forces are in our ODE and how the trajectory is progressing. These events are especially useful for determining failure conditions, such as a 'high kinetic interface with the ground', a suboptimal (and suborbital) result.

The optimizer, integrator, ODE, and force models are organized by a hierarchy of function calls. The user input file supplies the engine, rocket, and mission parameters, which are read into a params structure at runtime. At runtime, the optimizer generates a full control vector, which it passes to the integrator. The integrator queries the ODE, which returns the correct differential equations depending on the current state, which corresponds to the current stage. After generating a full trajectory using a variable-step Runge-Kutta solver, the integrator passes it back to the optimizer, which hands the trajectory off to an objective function that returns a feasibility score based on the constraint violations. Over hundreds of calls, the optimizer determines how each variable changes the feasibility score using a central finite difference scheme. The optimizer then converges over multiple iterations to a local minimum.

We use MATLAB's `fmincon` optimizer, a gradient based method designed to work on problems where the objective and constraint functions are both continuous and have continuous first derivatives. `fmincon` finds a local minimum of the objective function subject to nonlinear equality and inequality constraints, linear constraints, and variable bounds. The solver supports several algorithms. We use the Sequential Quadratic Programming (SQP) algorithm, which solves a quadratic subproblem at each iteration. Gradients of the objective and constraint functions are estimated using a central finite difference scheme, and iterations end when first order optimality or step size falls below tolerance. As a gradient based local solver, `fmincon` is not guaranteed to find the global minimum [3].

The optimization process is very sensitive to the first full control vector given it. Starting from an arbitrary point is unreliable, as the integrator frequently fails to establish an orbit for poor candidate solutions and cannot find a way to move the solutions, as all directions result in crashes, making the feasibility and objective fairly flat in this area. Because of the difficulty of the user estimating the appropriate initial guess, we employ a two-level optimization

architecture. The inner solver generates an initial guess by solving a reduced three variable problem, assuming the control nodes are zero throughout the flight. The constraints require the trajectory reach an apoapsis at the target orbit radius while carrying enough liquid propellant to circularize, as estimated by Tsiolkovsky's equation. Additionally, we require the flight path angle at atmosphere exit to be below a threshold value of 30° to ensure an effective gravity turn. This inner problem is much easier to solve, yet the constraints are tight enough to provide a reliable starting point for the main optimizer. Once the inner solution is found, the deflection nodes are estimated as a linear taper from the flight path angle at atmosphere exit to zero. This initialization reflects the objective of the tupper stage, which is primarily to increase tangential velocity for orbital insertion. This initial guess is then passed to the optimizer, which solves the full problem to convergence.

All design variables, objectives, and constraints are scaled prior to being passed to the optimizer. Proper scaling ensures that the finite-difference Jacobian approximations used by the SQP solver are computed with consistent step sizes across variables of very different magnitudes. Without it, a 1.0 (kg) change in propellant mass would have absolutely no effect compared to a 1.0 (radian) change in launch angle.

For candidates that reach an unphysical trajectory not covered by the objective function, we penalize them with a flat value of 10.0, chosen to be an order of magnitude larger than the starting scaled feasibility value of 1.0.

V. Results and Discussion

To run our optimizer, we first pick our target orbit, engine parameters, rocket parameters, and our setup values for the optimizer. The following values are rough estimates based off historical values.

% Target Orbit Parameters a = 6678100 e = 0.0	% Engine Parameters - Stage 1 totalTemp1 = 3500 totalPressure1 = 9000000 specificHeatRatio1 = 1.22 gasConstant1 = 355 Ae1 = 4.5 At1 = 0.15	% Rocket Parameters dragCoefficient = 0.25 referenceArea = 7.5 payloadMass = 5000 engineMass1 = 2500 engineMass2 = 800 massFrac1 = 0.1 massFrac2 = 0.1
% Planet Parameters radiusEarth = 6378100 massEarth = 5.97219e24 gravConstant = 6.6743e-11 atmoEndPercent = 0.000001	% Engine Parameters - Stage 2 totalTemp2 = 4000 totalPressure2 = 6000000 specificHeatRatio2 = 1.2 gasConstant2 = 600 Ae2 = 3.06 At2 = 0.0204	% Optimizer Setup maxThrustAngle = 1.57 gravityTurnVelocity = 200 numberOfControlNodes = 10 maxSimTime = 2000 massLowerBoundFactor = 0.01 massUpperBoundFactor = 10

The optimization process begins with the computation of an initial guess using the reduced problem formulation.

```

===== IG Result =====
Solid : 20812.6 kg (2.08 x scale)
Liquid : 6768.5 kg (0.68 x scale)
Angle : 31.78 deg (0.71 x scale)
TWR : 5.49
r_apo : 300.0 km
gamma_exit : 35.00 deg
delta_ig : [0.611 ... 0.000] rad
=====

```

When we feed this initial guess into our optimizer, the solution converges within our desired tolerances, although not within the constraints of the optimization function, in 31 iterations.

```

===== Optimization Result =====
Exit flag : -2
Iterations : 31
Func evals : 1030

```

Solid prop : 65620.7 kg
 Liquid prop : 15216.7 kg
 Total prop : 80837.4 kg
 Launch angle : 82.30 deg
 Delta nodes : 1.565 1.332 1.277 1.311 1.425 0.911 0.208 -0.729 0.326 -0.335
 Final orbit : a=299.4 km e=0.00173
 Target orbit : a=300.0 km e=0.00000

Plotting these results,

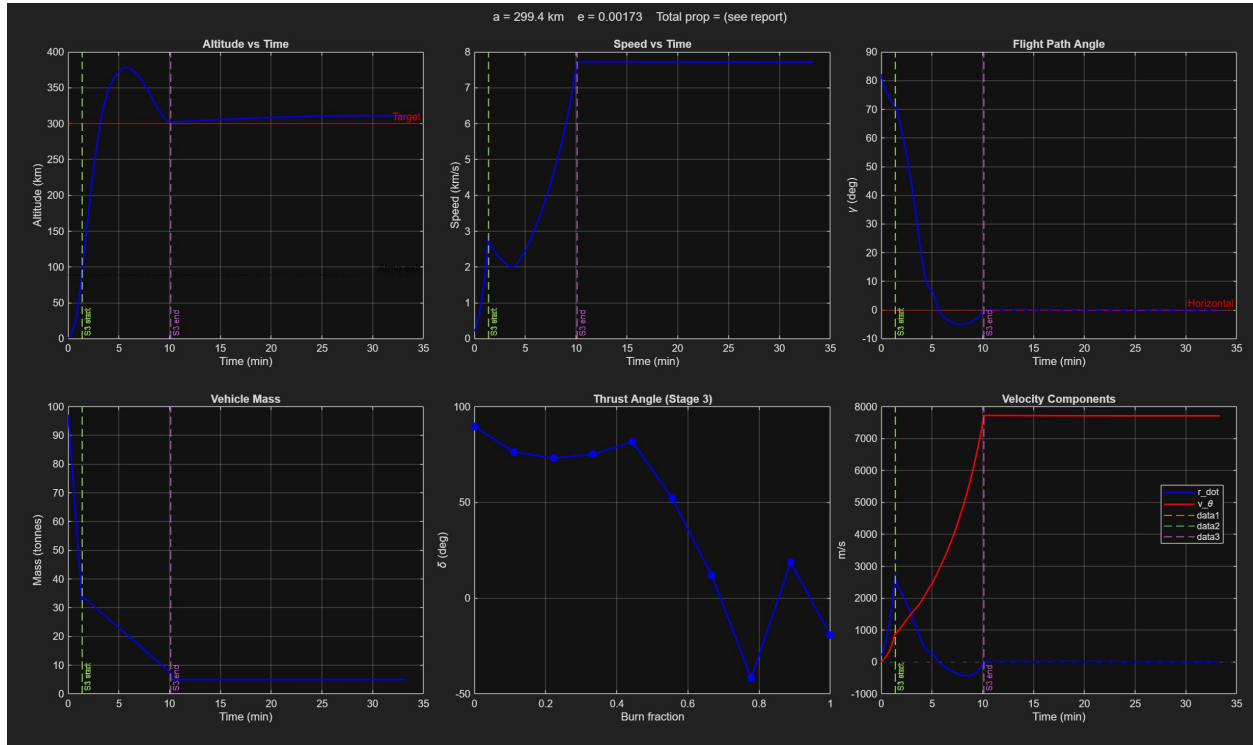


Figure 2: State Variables History

From the plots we can see that the solution has converged to a quick and high thrust followed by a slow and low thrust insertion burn. The trajectory overshoots the target altitude and falls back during the burn as the liquid stage increases tangential velocity. Initially, the upper stage burns downward to decrease radial velocity before burning tangential and circularizing. We can also plot the forces and velocities as components to see a similar result. Staging lines are drawn to indicate stage transition times.

It is important to recognize that these results are not necessarily the most optimal solution. These results represent a local minimum of the objective function that satisfies the imposed constraints and is dependent on the initial guess. If the initial guess is changed, or the boundaries modified, the optimizer can and does converge on a different solution. Notably, if the lower bound on the launch angle is lowered to allow a 60° launch angle, the optimizer converges to a solution in which the first stage undershoots the target altitude, while the second stage compensates during ascent.

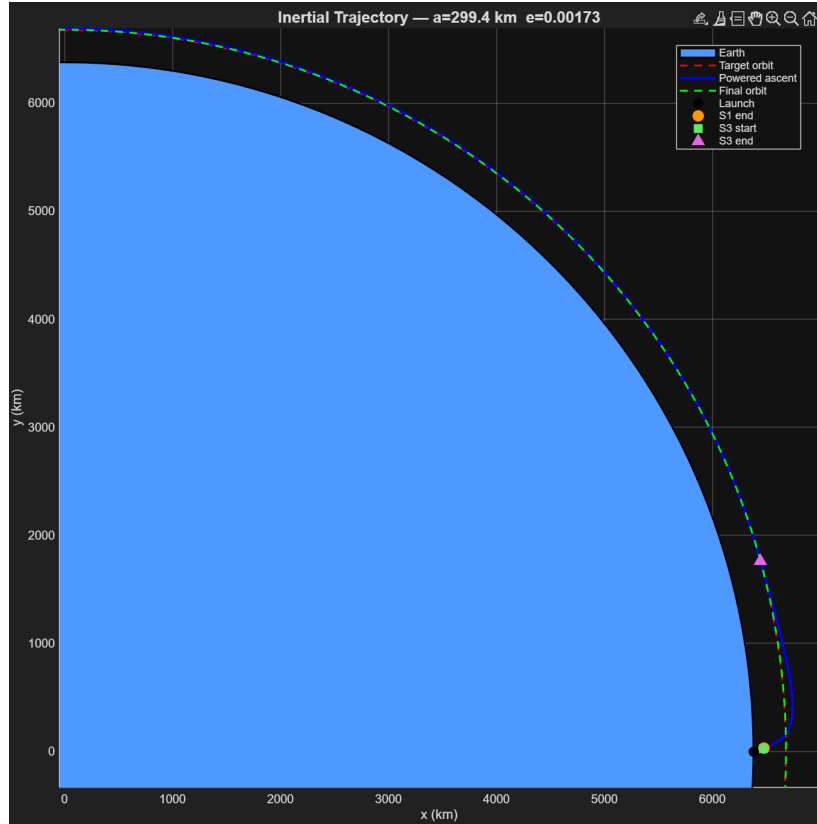


Figure 3: Cartesian Ascent Profile

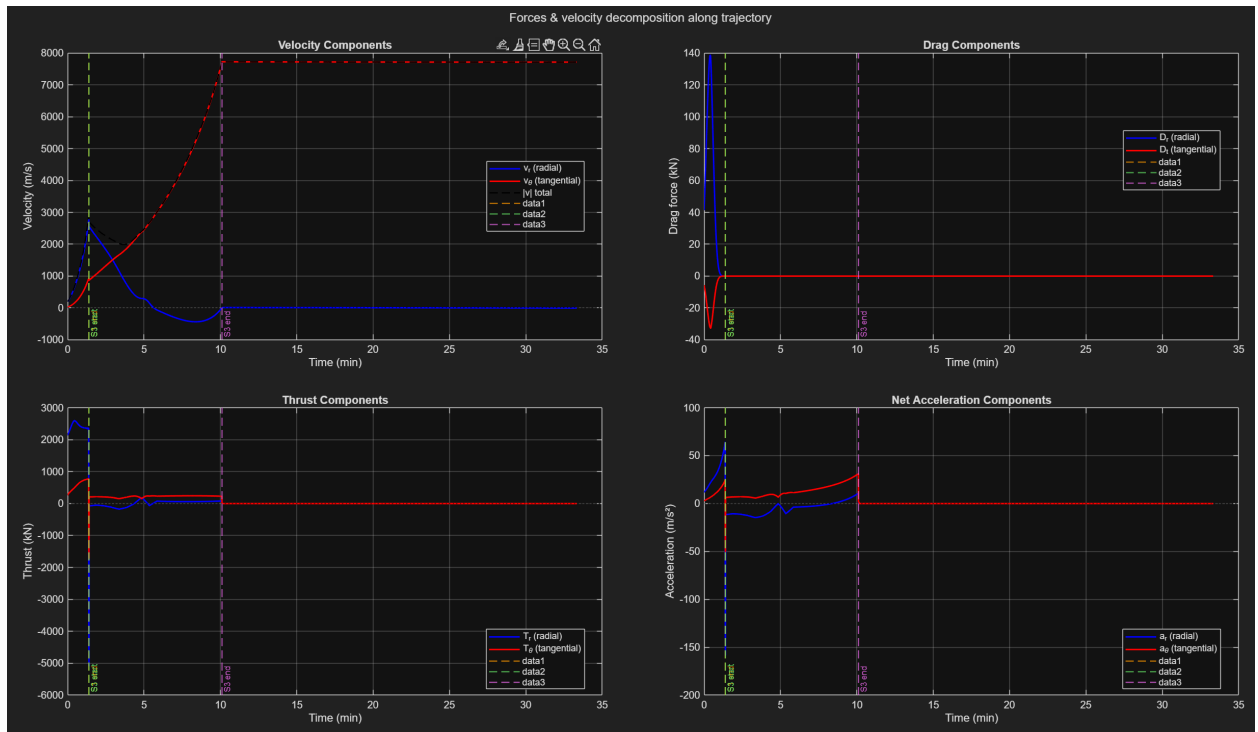


Figure 4: Velocity and Force Components History

The sensitivity of the solution to the initial conditions motivates further exploration of the solution space. As noted, fmincon converges to a local minimum, and the solution found is dependent on both the initial guess and the bounds imposed on the design variables. To investigate whether alternative trajectory strategies yield a lower propellant mass, the lower bound on the launch angle was relaxed to 55° , allowing the optimizer to explore a shallower initial trajectory. This changes the character of the gravity turn. A lower launch angle keeps the vehicle closer to horizontal during the atmospheric phase, leading to a higher drag exposure in exchange for a more tangential velocity at atmosphere exit. The optimizer was rerun from the same initial guess with this modified bound, converging to a qualitatively different solution.

=== Optimization Result ===

Exit flag : 2

Iterations : 44

Func evals : 1540

Solid prop : 58493.1 kg

Liquid prop : 16724.3 kg

Total prop : 75217.4 kg

Launch angle : 61.02 deg

Delta nodes : 0.197 -0.136 0.024 0.045 0.159 0.030 -0.034 0.057 0.013 0.037

Final orbit : a=290.8 km e=0.00026

Target orbit : a=300.0 km e=0.00000

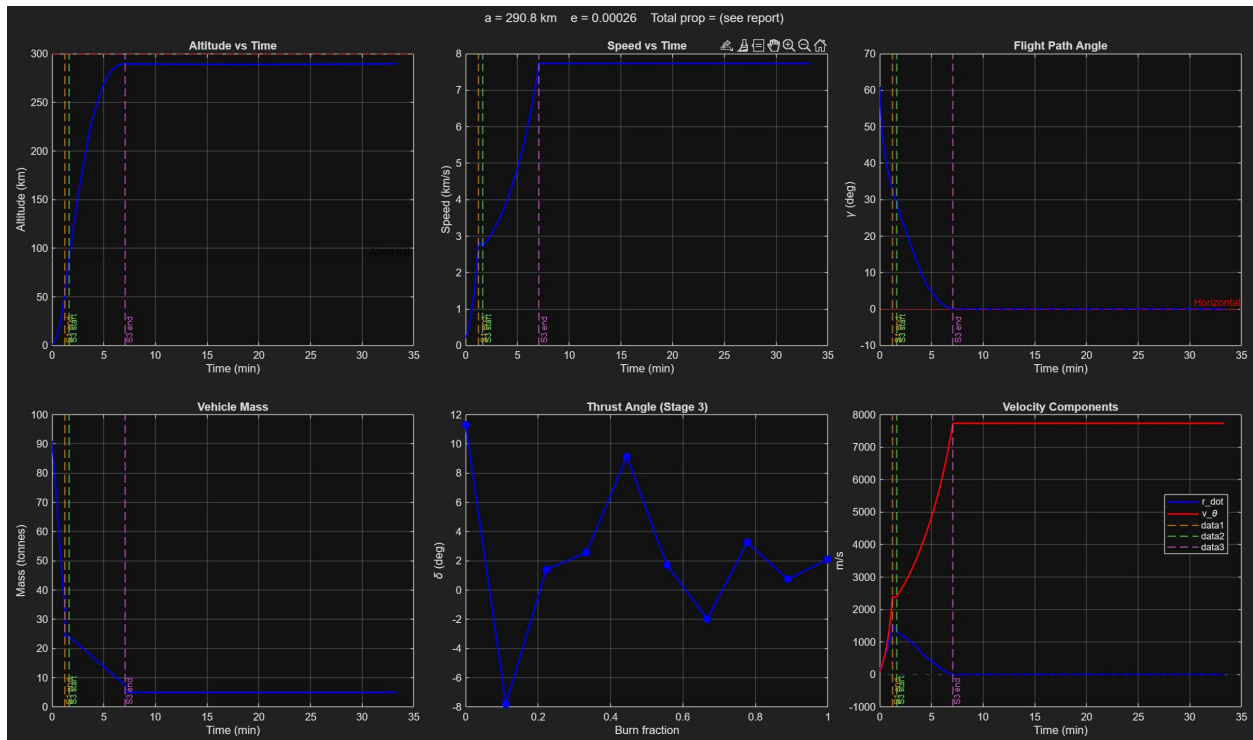


Figure 5: Second Solution State Variables History

This solution converges on a smaller orbit, indicating further refinement of the optimizer functions is necessary. The main difference for this trajectory is that it converges on a cheaper mass solution by decreasing the launch angle to 61° . Whether this solution is feasible depends largely on the vehicle's aerodynamics and attitude stability. A pitch over angle this aggressive is more consistent with low mass high thrust sounding rockets than larger, slower rockets.

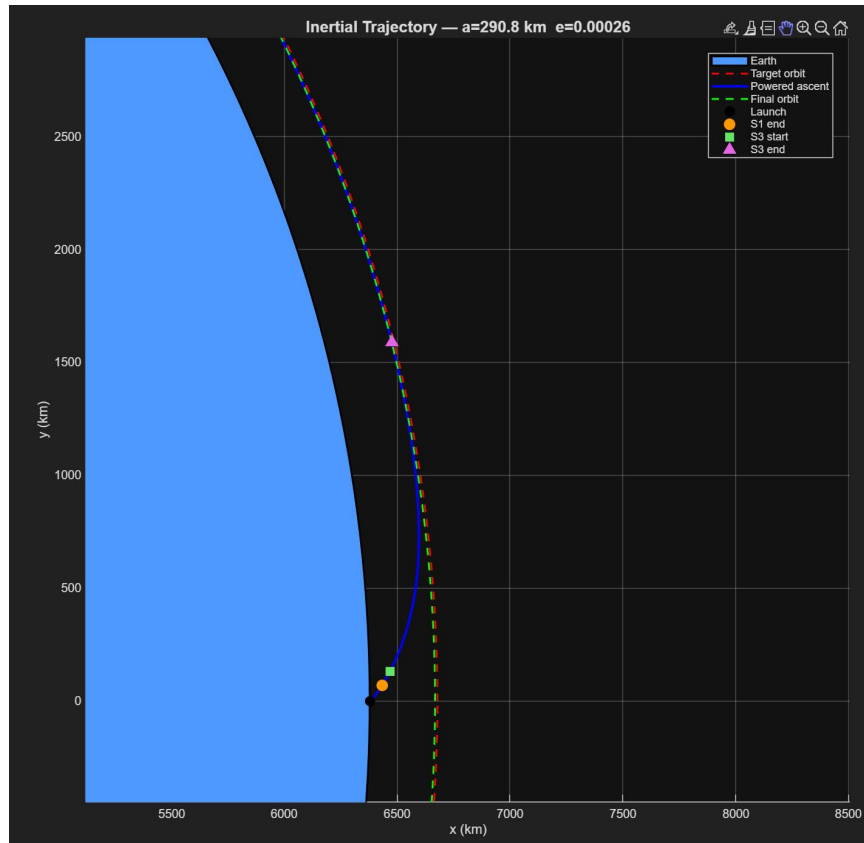


Figure 6: Cartesian Second Trajectory Plot

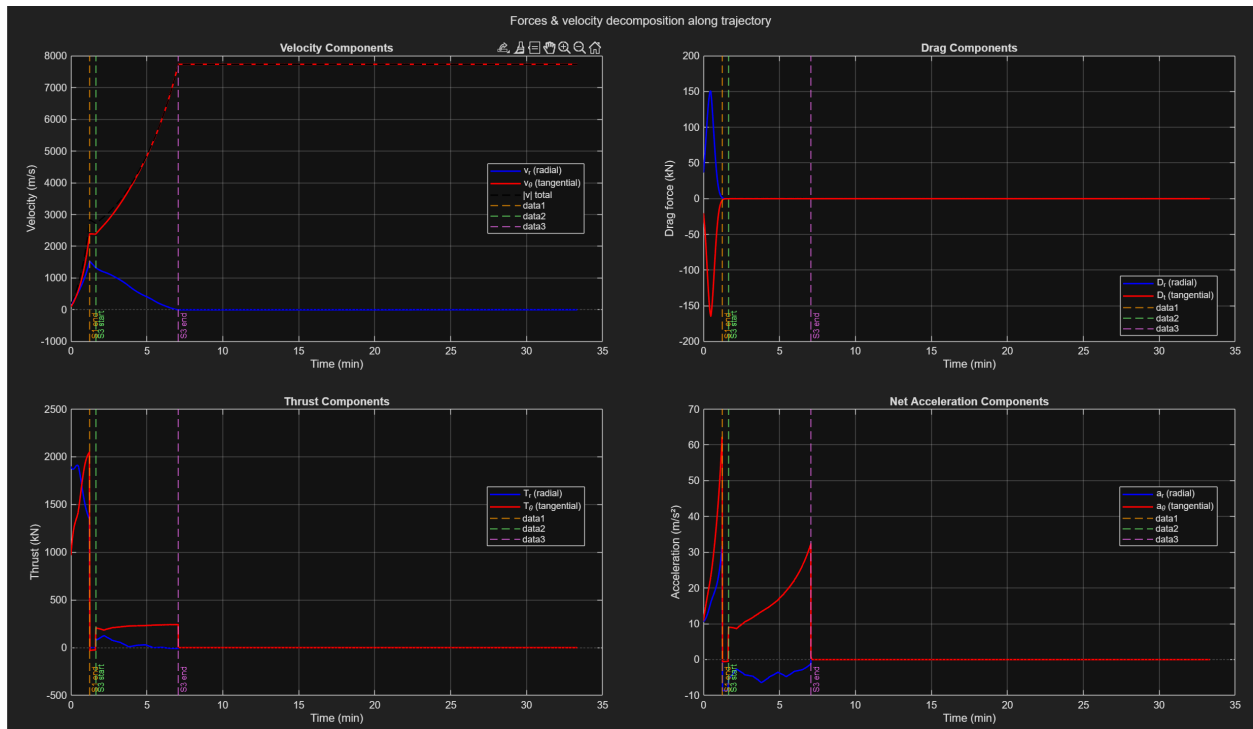


Figure 7: Velocity and Force Histories, Second Trajectory

VI. Conclusion

This study has presented a simplified framework for optimizing propellant allocation and ascent trajectory for a two-stage launch vehicle under fixed engine parameters. By forming the problem as a nonlinear program and solving by direct transcription, the model demonstrates how propellant mass distribution, launch angle, and thrust vector control collectively influence the ability to achieve a target orbit while minimizing total propellant consumption. Our results indicate the strengths and limitations of optimizers in general. While the solver converges to solutions that satisfy constraints, it is sensitive to initial guesses and generally finds the local, rather than global, minimum, especially for high dimension solution spaces. Tighter constraints and more precise objective functions can help solve this problem. Despite its simplifying assumptions, such as ideal engine performance and reduced atmospheric model, the framework captures the key physics behind ascent dynamics and provides insight into staging and trajectory design. The two-level optimization approach proved especially powerful in improving convergence reliability and offered a practical method for initializing these sorts of problems.

Future work should focus on increasing model fidelity and robustness. Incorporating more realistic aerodynamics and allowing additional control freedom such as variable ignition times and first stage thrust vectoring can help improve the model's realism, while refining the objective function to minimize gravitational and drag losses could improve accuracy.

Overall, this work demonstrates that even a simplified optimization framework can serve as a useful tool for preliminary launch vehicle design, allowing engineers to explore design sensitivities and trade-offs in trajectory planning.

Appendix

Appendix A: Derivation of Equations of Motion

Appendix B: Code Files

References

- [1] Humble, R. W., Henry, G. N., and Larson, W. J., *Space Propulsion Analysis and Design*, McGraw-Hill, New York, 1995.
- [2] The MathWorks, Inc., "ode45 — Solve nonstiff differential equations, medium order method," MATLAB Documentation, R2024b, Natick, MA, 2024, <https://www.mathworks.com/help/matlab/ref/ode45.html>.
- [3] The MathWorks, Inc., "fmincon — Find minimum of constrained nonlinear multivariable function," MATLAB Documentation, R2024b, Natick, MA, 2024, <https://www.mathworks.com/help/optim/ug/fmincon.html>.